

1.4 Das Entwurfsmuster Kompositum

Die Liste aus 1.3 funktioniert. Aber fast jede Methode muss Sonderfälle abfangen: das Ende der Liste (nachfolger == null) und die leere Liste (anfang == null). *Website: Station 1.4, Projekt Kompositum*

Aufgabe 1 Fallunterscheidungen zählen

a) Zähle in deiner Lösung von 1.3 die Abfragen auf null. b) Stell dir vor, am Ende der Liste stünde statt null ein echtes Objekt, der **Abschluss**. Was müsste er bei den einzelnen Methoden antworten?

Methode	== null in Liste	== null in Knoten	Antwort des Abschlusses
laenge()			
enthaelt(d)			
add(d)			
entfernen(d)			

Mit Abschluss gibt es nie mehr null: Die leere Liste besteht nur aus dem Abschluss.

Knoten und Abschluss sollen beide als nachfolger und als anfang stehen können. Sie brauchen also einen gemeinsamen Typ:

```
public class ListElement {
    public int laenge();
    public abstract boolean enthaelt(DatenElement d);
    public abstract ListElement add(DatenElement d);
    public abstract ListElement entfernen(DatenElement d);
    ...
}
public class Abschluss { ... }
public class Knoten extends ListElement {
    private DatenElement daten;
    private ListElement nachfolger;
    ...
}
```

Abstrakte Klasse und Kompositum

- Von einer **abstrakten Klasse** kann man keine _____ erzeugen. Ihre abstrakten Methoden haben keinen _____: Jede Unterklasse muss sie _____.
- Anders als ein Interface darf eine abstrakte Klasse auch _____ und fertige Methoden enthalten. Erben kann eine Klasse aber nur von _____ Klasse.
- Welche laenge() läuft, entscheidet das Objekt zur Laufzeit: _____.
- Kompositum**: Statt Fallunterscheidungen gibt es für jeden Fall eine eigene _____.

Aufgabe 2 Objektdiagramm

Zeichne alle Verweise ein: links eine leere Liste, rechts eine Liste mit Taxi 12 und Taxi 7.



Aufgabe 3 Projekt Kompositum, TODO 3a und 3b

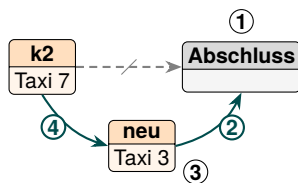
```
// Knoten
public int laenge() {
    return _____;
}
public boolean enthaelt(DatenElement d) {
    if (daten.istGleich(d)) {
        return true;
    } else {
        return _____;
    }
}
// Abschluss
public int laenge() { return _____; }
public boolean enthaelt(DatenElement d) { return _____; }
```

a) Ergänze und übertrage ins Projekt.

b) In Knoten.laenge() steht keine Abbruchbedingung mehr. Warum endet die Rekursion trotzdem?

Aufgabe 4 Projekt Kompositum, TODO 4a bis 4c

Beim Anhängen gibt es ein Problem: Der Abschluss kann den nachfolger seines Vorgängers nicht ändern. Lösung: Jedes add gibt zurück, **was danach an seiner Stelle steht**. Der Aufrufer trägt diese Rückgabe ein.



- ① Der Abschluss erzeugt einen neuen Knoten,
- ② trägt sich selbst als dessen Nachfolger ein
- ③ und gibt den neuen Knoten zurück.
- ④ k2 trägt die Rückgabe als Nachfolger ein und gibt sich selbst zurück.

```

// Abschluss (4a)
public ListElement add(DatenElement d) {
    Knoten neu = _____; // ①
    neu.setNachfolger(_____); // ②
    return _____; // ③
}

// Knoten (4b)
public ListElement add(DatenElement d) {
    nachfolger = _____; // ④
    return _____;
}

// Liste (4c)
public boolean add(DatenElement d) {
    anfang = _____;
    return true;
}

```

b) Warum muss auch die Liste das Ergebnis speichern (anfang = ...)? Denke an die leere Liste!

Aufgabe 5 Projekt Kompositum, TODO 5

```

// Knoten: Rückgabe = wer danach hier steht
public ListElement entfernen(DatenElement d) {
    if (daten.istGleich(d)) {
        return _____; // ich werde übersprungen
    } else {
        nachfolger = _____;
        return _____;
    }
}

// Abschluss
public ListElement entfernen(DatenElement d) {
    return this;
}

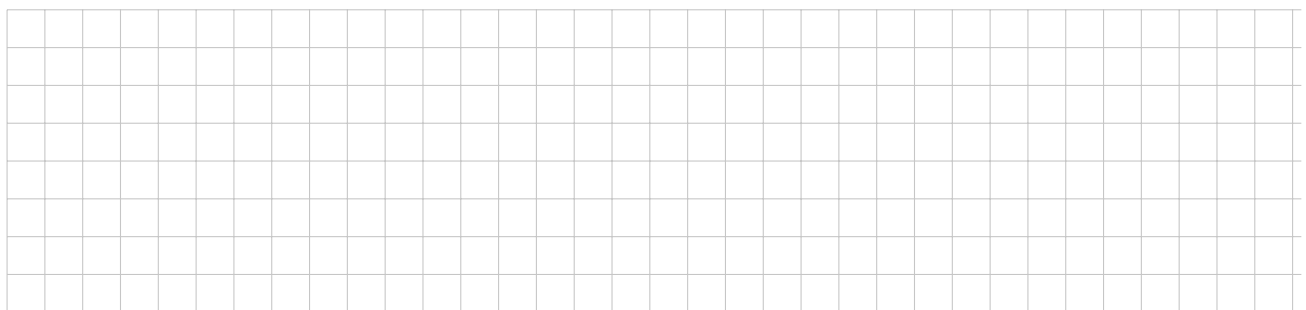
```

a) Ergänze Knoten.entfernen. Warum ist Abschluss.entfernen schon fertig?

b) Vergleiche mit 1.3. Was fällt weg?

Aufgabe 6 Klassendiagramm

Zeichne das Klassendiagramm (nur Klassennamen) mit Liste, ListElement, Knoten, Abschluss, DatenElement und Taxi: Assoziationen mit Namen und Multiplizitäten, Vererbung (durchgezogen, leeres Dreieck) und Implementierung (gestrichelt, leeres Dreieck).



Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- ListElement könnte auch ein Interface sein. Was spricht hier dafür, was dagegen?
- Jede Liste hat ihren eigenen Abschluss. Würde auch ein gemeinsamer Abschluss für alle Listen reichen?
- Knoten.add gibt this zurück, obwohl sich an seiner Stelle nichts ändert. Warum trotzdem?